The Definition of Dynamic Z

Robert Büssow Wolfgang Grieskamp
Frank Lattemann Eckard Lehmann

May 28, 1997 DRAFT

Contents

1 Introduction **2**

2 Example: Traffic Light Control System **5**
2.1 World Model 5
2.2 Safety Condition: First Try 6
2.3 Safety Property: Refinement 7

3 Example: Tempomat with Distance Control **7**
3.1 World Model 8
3.2 Defining the System Modes 9
3.3 Regulating the Speed 9
3.4 Putting it Together 11

4 Description of Dynamic Z **11**
4.1 Dynamic Values and Dynamic Schemes 12
4.1.1 Phases 12
4.1.2 Runs 13
4.1.3 Dynamic Values 13
4.1.4 Synchronicity 14
4.1.5 Dynamic Schemes 14
4.2 Dynamic Predicates 15
4.2.1 State Predicates 15
4.2.2 Transition Predicates 15
4.2.3 Plain Predicates as Dynamic Predicates 16
4.2.4 Chopping 16
4.2.5 Existence and Universality 17
4.2.6 Dynamic Quantification 17
4.2.7 Phase Number Constraints 17

4.2.8 Real-Time Constraints . . . 18
4.3 Embedding of Dynamic Schemes . . . 18
4.3.1 Declarations and Predicates . . . 18
4.3.2 Schema Text . . . 19
4.3.3 Expressions . . . 19
4.3.4 Schema Expressions . . . 19
4.4 Integration and Differentiation . . . 20
4.4.1 Integration . . . 20
4.4.2 Differentiation . . . 20
4.4.3 The Length of a Run . . . 20
4.5 Specialised Models . . . 21
4.5.1 Discrete Model . . . 21
4.5.2 Proportional Real-Time . . . 21

A Operational Specification **21**

B Embedding $\mu\mathcal{SZ}$ in Dynamic Z **22**

C Syntax and Semantics **22**

D Mathematics of Continuous Systems **23**
D.1 Numbers . . . 23
D.2 Metrics . . . 24
D.2.1 Metric Sets . . . 24
D.2.2 Convergency and Continuity . . . 24
D.2.3 Basic and Derived Metrics . . . 25
D.3 Linear Spaces . . . 26
D.3.1 Characterisation of Linear Spaces . . . 26
D.3.2 Basic and Derived Linear Spaces . . . 27
D.4 Integration and Differentiation over one Variable . . . 28

1 Introduction

We introduce *Dynamic Z*, an extension of Z by concepts of *temporal interval logics*. Dynamic Z serves for two purposes: on the specification-technical level, it shall improve the expressive power for the formulation of *invariants* over a system state, incorporating temporal resp. real-time aspects. On the level of foundation and implementation of tools for ESPRESS, it shall provide a unified calculus in which the combination of Statecharts and Z, $\mu\mathcal{SZ}$, can be embedded.

Motivation 1: Temporal Invariants. The Z notation provides a specification style for state-based systems which follows the pattern that over a global state which fulfils certain invariants operations are specified by relating pre- and post-states. While this pragmatics is elegant because of its simplicity – the logic of Z is "pure" – its restriction is in the expressive power for invariants: a state invariant can only describe a static property, which has to hold for every state, but it cannot constraint the admissible *evolutions* of states.

Consider the requirements specification of a (shallow-abstraction) of a light-traffic system (LTS). Lets assume the *functionality* of the LTS is given in the usual operational Z style[1]: there is a specification of a global state, incorporating sensors and actuators, and there are operations which specify legal transitions over this state. Let this specification be complete in the sense that the operations describe all the needed behaviour of the LTS. However, since the LTS is a safety-critical system, the specification must render clearly that certain *safety-conditions* are satisfied, which is usually achieved in the Z approach by postulating invariants on the system state. Informally, the basic safety invariant for the LTS is the following:

> If a traffic flow is open, then any possible conflicting flow must be closed at least the last t seconds.

This condition employs a *temporal* aspect, which demands that a certain condition (a flow is open, i.e. is signalled green) is only legal if a certain other condition (a flow is closed, i.e. is signalled red) has been present for some time. Plain Z doesn't support adequately the specification of such invariants. Though it is in principle possible to encode the above condition by introducing an auxiliary variable "remembering" the duration of signals, the resulting specification would be cumbersome and error-prone because of its redundancy.

Motivation 2: Unified Calculus. ESPRESS aims at a *model oriented* specification style meaning that an operational model is to be constructed for the software under consideration which is suitable e.g. for simulation. In the ESPRESS methodology, safety properties as the one sketched for the LTS above, as well as other behavioural properties have to be a consequence of an operational Statecharts&Z model:

$$\textit{Statecharts\&Z Model} \vdash \textit{Safety\&Behavioral Properties}$$

In order to give a deduction-technical foundation of this relation which enables automatic tool support, it is desirable to have a *unified calculus* which allows to embed a Statecharts&Z model and the properties it has to obey in a single framework.

It is believed that *temporal interval logics* provides an adequate approach for this purpose. Such a framework should base on standard predicate logic, which temporal logics generally does. The version of *interval* logics additionally supports the description of operational aspects as they come from Statecharts, such as the sequential composition of behaviours.

[1] Respectively in $\mu\mathcal{SZ}$, the combination of Statecharts and Z as used in the ESPRESS project

Motivation 3: Capturing Hybrid Systems. In order to preserve the possibility that the techniques and tools developed in ESPRESS survive behind the limited duration of the project, a unified calculus for the ESPRESS languages Statecharts, Z, and temporal logics should be powerful enough to capture concepts which at the moment don't have a high priority on the agenda of planned support in ESPRESS, but which are known to be nevertheless important – as it is the case for continuous techniques, which are apparently necessary for an adequate specification of certain aspects in the ESPRESS case study "Fahrtgeschwindigkeitskonstanter" (FGK). This extended functionality should not, however, complicate the basic model if it is not needed.

Hybrid techniques provide the necessary concepts for this purpose, as we believe. In an hybrid setting, discrete and continuous modelling techniques can coincide. Phase transition systems, as proposed as a generic model for hybrid systems, provide the necessary link between hybrid automatons and hybrid Statecharts and a unified treatment in temporal interval logics.

Outline of Dynamic Z Dynamic Z bases on two central notions: that of a *dynamic value* and that of a *dynamic schema*. A dynamic value models a trace or *run* of a certain entity over time. A dynamic schema collects several of these values and enforces them to behave *synchronously* under constraints described by a predicate of temporal interval logics, called a *dynamic predicate*.

A major intention of Dynamic Z is to be compatible with Z, and to keep the language extensions as moderate as possible. This is achieved by treating both dynamic values and dynamic schemes as "first-order citizens" of Z: semantically, a dynamic value is just an ordinary value of a certain structure, and a dynamic schema represents just an ordinary set of bindings of dynamic values which happen to behave synchronously. Henceforth, the standard language repertoire is available for dynamic values and schemes. The schema calculus can be used as usual for the structuring of specifications via schema inclusion and schema reference, and it is possible to describe sets, relations and functions which work on dynamic values.

Dynamic Z provides specialised models, *discrete* and *real-time proportional*, which can be selected for a specification by inclusion of according axioms. The non-discrete models support the specification of values which behave discrete as well, and are thus hybrid.

Dynamic Z provides a small set of operators which allow to express operational and concurrent behaviour. These operators can be used to represent Statecharts or other formalisms for concurrency, such as e.g. CSP, in Dynamic Z. The operators reflect the basic operational concepts of *computation*, *choice* and *preemption*.

Outline of this Paper This paper doesn't intents to give a tutorial introduction into Dynamic Z– it is rather a systematic description of the full language (Section 4), which is accompanied by two introducing examples (Section 2 and Section 3). In particular, we describe the general case of the hybrid model of Dynamic Z, though for applications which only require the discrete sublanguage not all definitions are required.

However, the conceptual distance between the discrete language and the full hybrid language isn't as large as to be expected on a first sight – as long as the reader has some intuitive understanding of concepts of the infinitesimal calculus, such as convergency, continuity, ex-

istence of limes, integration and differentiation. These concepts will be formalised (in Z) for the interested reader in Appendix D.

A formal semantics of Dynamic Z is not yet provided. The major problem here is the formalisation of the operational temporal operators computation, choice and preemption (which description is, due to their experimental character, evacuated to Appendix A). A formalisation of the other operators is fairly standard, but the operational ones provide some open problems according to the reflection of deadlock and divergency in the semantic model.

2 Example: Traffic Light Control System

As an introducing example to Dynamic Z, we specify the safety property of the TLS as mentioned in the introduction. For this purpose, we first formalise the basic *terminology* of a (rather simplified) world-model of the TLS. We then give two versions of the safety-property: a simplified one, which exactly reflects what has been stated in the introduction, and a bit more complicated version, which reflects the actual situation in the TLS case study.

It should be noted that the specification represented in this Section doesn't makes explicite use of the hybrid capabilities of Dynamic Z. However, it makes use of arbitrary dense real-time constraints, which can be be modelled in the discrete as well as the continuous time model.

2.1 World Model

The TLS controls a set of signal heads by switching its lamps on and off. The set of signal heads are partitioned into signal groups, which are associated with traffic flows of pedestrians and vehicles. In order to concentrate on the illustration of Dynamic Z, we abstract here from these details, and just represent entire signal-groups by the given type

$$[SignalGroup]$$

A signal group may be either closed or open, a status modelled by the free type

$$SignalStage ::= open \mid closed$$

From the layout of a concrete TLS installation we assume a relation to be given which describes those signal groups which are in conflict, meaning that the traffic flow they control share the same space in the junction. Obviously, the *conflict* relation has to be non-reflexive and symmetric:

$$conflict : SignalGroup \not\supseteq SignalGroup$$

$$\forall\, sg_1, sg_2 : SignalGroup \bullet (sg_1, sg_2) \in conflict \Rightarrow ((sg_2, sg_1) \in conflict \land sg_1 \neq sg_2)$$

The layout specification of a concrete TLS further induces the so-called "inter-green-time", which exactly defines how long a traffic flow must be closed to clear the road until the next traffic can be opened for a particular conflicting pair of signal-groups. The inter-green-time is defined by a function mapping conflicts to time durations:

$inter_green_time : conflict \not\subseteq TIME$

The (much simplified) "world state" of the TLS now consists of just one variable which maps signal groups to their current stage:

DYNAMIC LTSWorld

$stage : \mathrm{dyn}(SignalGroup \not\subseteq SignalStage)$

The above schema definition introduces a so-called *dynamic schema* containing a so-called *dynamic value*, *stage*. At this point, it is sufficient to understand the dynamic schema *LTSWorld* just as a normal schema which introduces a state variable *stage* whose value at each point of time is of type $SignalGroup \not\subseteq SignalStage$. The type constructor dyn A which is used to declare the type of *stage* can usually be omitted, in which case it will be inserted implicitly; it is shown up here in order to avoid a misinterpretation for future usages of dynamic schemes throughout this paper. We will give more explanations on the role of dynamic schemes and dynamic values in Section 3. Pragmatically, the main difference of dynamic schemes is in the power of the language for specifying invariants, as will be seen below.

2.2 Safety Condition: First Try

We now specify the safety condition from the introduction, which reads:

> If a traffic flow is open, then any possible conflicting flow must be closed at least the last t seconds.

This is specified as follows:

$t : TIME$

DYNAMIC LTSSafety

LTSWorld

$$\forall sg_1, sg_2 : SignalGroup \mid (sg_1, sg_2) \in conflict \bullet \forall at : TIME \bullet {}^{at}\overline{\lceil stage\ sg_1 = open \rceil} \Rightarrow {}^{at-t}\overline{\lceil stage\ sg_2 = closed \rceil}^{at}$$

This invariant makes use of the following temporal formulas, which we call *dynamic predicates*:

- The dynamic predicate $\lceil \phi \rceil$ is satisfied for intervals in which the normal Z predicate ϕ is satisfied continously.
- The dynamic predicate $\overline{\delta}$ comes in several variants. Here we have used the following forms:
  - ${}^{t}\overline{\delta}$ holds for those interval where at some so-called "time reference point" a subintervals starts for which δ holds.
  - ${}^{t_1}\overline{\delta}^{t_2}$ holds for those interval where the subinterval for which δ holds starts at reference point t_1 and ends at t_2.

2.3 Safety Property: Refinement

The refined safety condition reads as follows:

> For each pair of conflicting signal groups $(sg_1, sg_2) \in \mathit{conflict}$ and each observed interval; if sg_1 is opened at the beginning of the interval and sg_2 is opened at the end of the interval, then both have to be closed for at least $\mathit{inter_green_time}(sg_1, sg_2)$ in between the interval.

This is formalised as follows:

DYNAMIC LTSSafetyRefined

LTSWorld

$$\forall\, sg_1, sg_2 : \mathit{SignalGroup} \mid sg_1 \ \underline{\mathit{conflict}}\ sg_2 \bullet \\ \Box \left(\frac{\vdash}{\lceil sg_1 \in \mathit{opened} \rceil} \setminus \frac{}{\lceil sg_2 \in \mathit{opened} \rceil \dashv} \Rightarrow \frac{\mathit{inter_green_time}(sg_1, sg_2)}{\lceil sg_1 \in \mathit{closed} \wedge sg_2 \in \mathit{closed} \rceil} \right)$$

This condition makes use of the following dynamic predicates:

- The dynamic predicate $\Box\delta$ is satisfied for intervals in which the dynamic predicate δ holds for any sub-interval.
- $\frac{\vdash}{\delta}$ and $\frac{}{\delta \dashv}$ denote the intervals where δ holds at beginning resp. the end of the interval.
- $\frac{l}{\delta}$ denotes the intervals which contain a sub-interval for which δ holds the duration l.
- $\delta_1 \setminus \delta_2$ is the famous chop operator from interval logics, and holds for those intervals which can be chopped at a point, such that δ_1 holds from the beginning up to the "left" of this point, and δ_2 from the "right" of this point up to the end. (The notions of "left" and "right" require some deeper understanding, and will be discussed in Section 4. In a discrete model – which is sufficient for the problem of the LTS – we can think of the chopping point as a transition from one state to the next, and left and right of this point address just observations at consecutive discrete states.)

3 Example: Tempomat with Distance Control

As a larger example which illustrates Dynamic Z, a shallow abstraction of the problem of a *tempomat with distance control* is considered, which will be called FGK for short in the sequel[2]. This component of a car system serves to automatically regulate the speed according to a drivers wish. In addition to this classic functionality of a tempomat, the FGK recognises other cars driving ahead, adapting the regulated speed appropriately.

In contrast to the TLS example, where we have only considered a selected safety property, for the FGK we describe safety as well as behavioural requirements in a nearly complete way

[2]FGK is an abbreviation for the German notion "Fahrtgeschwindigkeitskonstanter".

(of course, relative to the course abstraction of the problem we have chosen). The only aspect of the FGK which is not fixed is the speed regulator, which will be only loosely determined.

The FGK requirements make use of the hybrid capabilities of Dynamic Z, since they rely on differentiation and integration of observed values.

3.1 World Model

The variables representing the relevant view on the "real-world" for the FGK problem are collected in the schema *FGKWorld* below. The free type *FGKControl* describes possible control actions of a driver:

$$FGKControl ::= start \mid stop \mid none$$

DYNAMIC FGKWorld

$$actualSpeed : \mathrm{dyn}\, SPEED$$
$$distanceAhead : \mathrm{dyn}\, LENGTH$$
$$control : \mathrm{dis}\, FGKControl$$

Here, the variable *actualSpeed* defines the actual speed of the car, *distanceAhead* the distance to a car driving ahead, and *control* the actions a driver performs. For reasons of simplicity it is assumed that the case that no car drives ahead is represented by an appropriate large value of *distanceAhead*. The meaning of the values of *control* will be explained in the next Section.

In general, the above schema definition plays pragmatically the same role as a normal Z schema definition which is used to introduce the variables of a system state. However, there are two differences: first, the variables are declared to be *dynamic values* over sets *SPEED*, *LENGTH* and *FGKControl*, by use of the set constructors $\mathrm{dyn}\, A$ resp. $\mathrm{dis}\, A$. Secondly, the schema which collects them is marked as being a *dynamic schema*.

Intuitively, a dynamic value is just a value which changes over time. A dynamic schema bundles a collection of such values and enforces them to behave *synchronously*, enabling the possibility to pointwise relate their behaviour by a single property given in temporal interval logics. Above, no property has been necessary. The diverse forms of properties will be introduced in the next Sections.

Mathematically, a dynamic value is a function over time, where the notion of time is structured in order to support the coexistence of continuous and discrete behaviour. More precisely, a dynamic value is represented by a so-called *run*, where a run is a finite or infinite sequence of so-called *phases*, and a phase is a continuous function over on open time-interval. Continuous computation takes place inside of phases, and discrete computations on transitions from one phase to the next. If a value is constant during a phase and only changes on transitions, it is called discrete. In the above example, the variable *control* is of this kind, which has been expressed by using the set constructor $\mathrm{dis}\, A$ in the declaration, which represents the subset of dynamic values $\mathrm{dyn}\, A$ over some set A which behave discrete[3].

[3]In fact, the set constructor $\mathrm{dis}\, A$ is redundant here – functions over incoherent sets such as *FGKControl* can by definition only be continuous if they are constant.

3.2 Defining the System Modes

The FGK may be either in the mode passive or active. Each of the modes begins with one of the control actions *start* or *stop* and extends as long as the control action *none* is present, which models that the driver doesn't performs any action. This is expressed by dynamic schemes as follows:

DYNAMIC FGKActive

$$FGKWorld$$

$$\lceil control = start \rceil \setminus \lceil control = none \rceil$$

DYNAMIC FGKPassive

$$FGKWorld$$

$$\lceil control = stop \rceil \setminus \lceil control = none \rceil$$

The runs of e.g. *FGKActive* are specified to start with phases where $control = start$ holds and continue with phases in which $control = none$ is satisfied. Technically, the following constructs are used to specify this behaviour:

- A dynamic predicate such as $\lceil \phi \rceil$ is satisfied for runs in which the predicate ϕ is satisfied continously during each phase. It is called a *state predicate*.
- The dynamic predicate $\delta_1 \setminus \delta_2$ is satisfied by those runs which can be splitted into sub-runs such that δ_1 holds for the first and δ_2 for the second sub-run.

3.3 Regulating the Speed

If the FGK is active, the speed of the car should converge against a nominal speed, which is derived by either a drivers demand or by the speed needed to hold a certain safety distance to a car driving ahead.

In order to specify this behaviour, first an auxiliary variable is introduced which holds the value of the speed demanded by the driver. This speed is set from the actual speed at the moment the driver starts the FGK; it keeps constant during the active mode. This is expressed as follows:

DYNAMIC FGKActiveDemanded

$$FGKActive$$

$$demanded : \mathrm{dis}\, SPEED$$

$$\lceil demanded = actualSpeed_{\triangleleft} \rceil$$

In a dynamic predicate $\lceil \phi \rceil$, a name decorated as $x_{\triangleleft}$ refers to the value of the dynamic value at the beginning of the run for which the dynamic predicate holds.

The nominal speed of the car is now defined to equal either to the drivers demand or to a speed necessary to hold a safe distance to a car driving ahead. This distance is calculated by the function *safeDistance* in dependency of the actual speed, the specification of which is left open (in praxis it would be derived according to legal regulations):

$$safeDistance : SPEED \not\subseteq LENGTH$$

DYNAMIC FGKActiveNominal

$FGKActiveDemanded$
$nominalSpeed : \mathrm{dyn}\ SPEED$

$\lceil \mathbf{if}\ distanceAhead < safeDistance\ actualSpeed$
$\ \mathbf{then}\ nominalSpeed = actualSpeed + \mathbf{diff}\ distanceAhead$
$\ \mathbf{else}\ nominalSpeed = demandedSpeed \rceil$

In a dynamic predicate $\lceil \phi \rceil$, an extended set of expressions is available – here, the differential quotient of an expression ω, **diff** ω, has been used to denote the relative speed to the car driving ahead.

Finally, the effect of the speed regulation is specified by establishing loose constraints on the evolution of the absolute difference of nominal and actual speed. It is demanded that the mean value of the difference over a time window must decrease in time, that is, its differential quotient must be less then 0:

$$window : TIME$$

DYNAMIC FGKActiveRegulate

$FGKActiveNominal$

$\lceil \ell > window \Rightarrow \mathbf{diff}\ (\frac{\int_{\mathrm{window}} (\mathrm{abs}(nominalSpeed - actualSpeed)))}{window}) < 0 \rceil$

The property of the schema definition above makes use of the following special expression in a state predicate $\lceil \phi \rceil$:

- The special expression ℓ in a state predicate denotes the length in time of a run for which the state predicate holds upto the current point of observation. It is actually an abbreviation for $\int 1$ (see below).
- The special expression $\int \omega$ denotes the integral of the expression ω from the beginning of the run up to the current point – that is mathematical the bounded integral $\int_0^\ell \omega$. The variant $\int_t \omega$ allows to integrate over a time window from some point in the past up to the current point – it denotes mathematical the bounded integral $\int_{\ell-t}^{\ell} \omega$, which is defined as an expression only if $\ell > t$ holds.

3.4 Putting it Together

The overall FGK requirements can now be defined as a composition of the schemes *FGK-ActiveRegulate* and *FGKPassive*. The FGK just repeatedly switches between these modes:

DYNAMIC FGKRequirements

$$FGKWorld$$

$$((FGKActiveRegulate \not\leq FGKWorld) \vee FGKPassive) \setminus FGKRequirements$$

Following technical remarks:

- A reference to a named dynamic scheme in a dynamic predicate behaves as to be expected in a Z confirming framework: the property of the schema can be thought of being textually expanded at the reference point.
- In Dynamic Z resp. the Z of *ESZ*, schema references in the property part may be recursive. This has been used to express the repetition of the modes active and passive.
- The formula $\delta_1 \vee \delta_2$ holds for runs which satisfy δ_1 or δ_2. Note that by definition, the active and the passive modes exclude each other.

4 Description of Dynamic Z

In this Section, we provide a systematic description of Dynamic Z. We first introduce the notions of dynamic values and dynamic schemes. We then define dynamic predicates, which are used to specify the property of a dynamic schema. It follows a description of the embedding of dynamic schemes in the Z language. Finally, we discuss the integration and differentiation operators in dynamic predicates, as well as specialised models which can be obtained for Dynamic Z by adding global axioms to a specification, for example, enforcing that a specification uses a discrete model. The operational predicates, which are intended to represent e.g. Statecharts in Dynamic Z, are found in Appendix A.

The description style used throughout this section is a mixture of informal and formal techniques. As will be seen, many of the language elements of Dynamic Z can be introduced as plain Z declarations. In the other cases, we just introduce syntactic sugar, which can be reduced to plain Z.

mathematically a sequence of continuous functions over an open interval of dense time. The sequences are called *runs*, and their elements, the continuous functions, *phases*. If phases are constant mappings, a classical discrete model is obtained as a special case. The time notion used by phases does not necessarily coincides with real-time – if it does, a *proportional* time-model is yielded.

4.1 Dynamic Values and Dynamic Schemes

A dynamic value is represented by a so-called *run*, which is an abstraction of a sequence of so-called *phases*. A phase models an observation of a value for a certain real-time duration[4]. Whether we look at the special case of a discrete or the general case of an hybrid model depends on the exact properties of phase. Actually, the hybrid notion of phases contains the discrete one, but we will also sketch how a "standalone" model of Dynamic Z can be defined which only supports discrete phases..

Dynamic schemes represent groups of dynamic values which behave synchronously. Two runs are said to be synchronous, if they contain the same number of phases which pairwise have the same duration. A group of synchronous runs is isomorphic to a single run which is build from the cartesian product of the values at each point; however, the first representation is more suitable for a smooth integration with Z, since a dynamic schema then is just a plain schema with the synchronicity constraint on its values.

4.1.1 Phases

In a discrete model, a phase over A, phase A, is just pair of a value and of a positive duration:

$$\text{phase}_{discrete}\, A == A \times \{d : TIME \mid d > 0\mathsf{s}\}$$

The set *TIME* describes the set of reals weighted with the physical unit of seconds.

The hybrid model simply generalises from this definition by allowing the value to continuously change during a phase. A phase over A is then a continuous function from an open interval of dense time into A, which is defined in Z as follows:

$$\text{phase}\, A == \{f : TIME \not\rightarrow\!\!\!\!\!\!\circ\;\; A \mid \exists\, b : TIME \mid b > 0\mathsf{s} \bullet \mathsf{dom}\, f = (0\mathsf{s}, b)\}$$

The phase function has its own relative time, starting at 0s (this phase-relative time is only part of the semantic model, and not directly observable on the specification level.). The set $(0, b)$ denotes an open interval from *TIME*. The function-space constructor $A \not\rightarrow\!\!\!\!\!\!\circ\;\; B$ denotes the set of *continuous functions* from A to B[5].

A phase function which represents a *constant* mapping is obviously isomorphic to the representation phase_{discr} – thus the model of discrete dynamic behaviour is naturally contained as a special case in the more general model of continuous dynamic behaviour. By definition of continuity, phases over incoherent, "discrete" sets (such as e.g. phase $\gtrsim$) *must* be *constant* functions.

Since the domain of a phase is an open interval and since a phase is a continuous function, the *limes* of a phase against the left and right borders of a phase does always exists:

$$\mathsf{left}[A] == (\lambda\, p : \text{phase}\, A \bullet \lim_{\rightarrow 0}\, p)$$
$$\mathsf{right}[A] == (\lambda\, p : \text{phase}\, A \bullet \lim_{\rightarrow min\{b : TIME \mid \forall\, t : \mathsf{dom}\, p \bullet t < b\}}\, p)$$

[4]The duration of a phase represents an *internal* time measure, which does not necessarily coincides with external time. Thus, from a point of an external observer, the duration of a phase may be zero, enabling the possibility of instantaneous behaviour as required for Statecharts. See Section 4.5.

[5]By definition we only allow functions for which the limes is defined in every point.

As long as only discrete values are concerned, the values at the left and right border of a phase are identical, since the phase function is constant.

4.1.2 Runs

A run over A, run A, is a function $\gtrsim \nleqslant$ phase A, whose domain is downwards-closed:

$$\mathrm{run}\, A == \{f : \gtrsim \nleqslant \mathrm{phase}\, A \mid \forall\, i : \gtrsim \bullet i + 1 \in \mathsf{dom}\, f \Rightarrow i \in \mathsf{dom}\, f\}$$

Thus runs are finite or infinite sequences of phases[6]. For each $i, i+1 \in \mathsf{dom}\, r$, where r is a run, the pair $(\mathsf{right}(r\ i), \mathsf{left}(r\ (i+1)))$ is called a *phase transition*. A phase transition models a timeless, spontaneous change of a dynamic value. Discrete values can *only* change during phase transitions. Continuous values *may* change during a phase transition, representing a leap in the continuous behaviour, if $\mathsf{right}(r\ i) \neq \mathsf{left}(r\ (i+1))$. Figure 1 shows an example of run with two dynamic values X and Y, which behave synchronous.

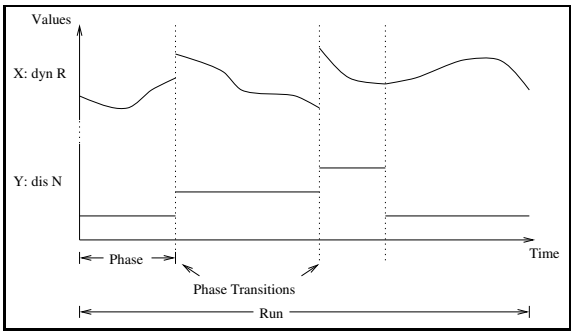


Figure 1: Run with two dynamic values

Because a run is a kind of encoding of a sectionally continuous bounded function, *integration* over a dynamic value is always a total operation. In contrast, *differentiation* is generally a partial operation, since continuity only ensures the existence but not the uniqueness of the limes of the differential quotient.

4.1.3 Dynamic Values

A dynamic value over A, dyn A, is just an abstraction of a run, which is defined by a free type. This abstraction serves to avoid clashes with other set-kinds in Dynamic Z, which incidentally have the same structure as runs:

$$\begin{aligned} &\mathit{dyntype}[A] ::= (_\mathsf{dyn})\lesssim \mathrm{run}\, A \gtrsim \\ &\mathrm{dyn}\, A == \mathit{dyntype}[A] \\ &(_\mathsf{run})[A] == (_\mathsf{dyn})^{\sim} \end{aligned}$$

The subset dis A of dyn A contains those runs where the value is constant during phases, and only changes on phase transitions:

[6]Standard sequences of Z are finite, and hence cannot be used here.

$$\mathrm{dis}\, A == \{d : \mathrm{dyn}\, A \mid \forall\, p : \mathsf{ran}(d_{\mathsf{run}}) \bullet \exists\, x : A \bullet \mathsf{ran}\, p = \{x\}\}$$

Further subsets of dynamic values can be easily introduced in Dynamic Z itself (see for example **??**).

4.1.4 Synchronicity

Synchronicity is the condition which makes it possible to relate the behaviour of different dynamic values. Two dynamic values are said to be synchronous, if their runs contain the same number of phases and the domains of the phases are pairwise identical:

$$\begin{array}{l}(_ \bowtie _)[A, B] = \\ \quad \{x : \mathrm{dyn}\, A;\ y : \mathrm{dyn}\, B \mid \\ \qquad \mathsf{dom}\, x_{\mathsf{run}} = \mathsf{dom}\, y_{\mathsf{run}} \land (\forall\, s : \mathsf{dom}\, x_{\mathsf{run}} \bullet \mathsf{dom}(x_{\mathsf{run}}\, s) = \mathsf{dom}(y_{\mathsf{run}}\, s))\}\end{array}$$

The two dynamic values in 1 are synchronous.

4.1.5 Dynamic Schemes

A dynamic schema has the general form $\not\equiv x_1 : \mathrm{dyn}\, A_1;\ \ldots;\ x_n : \mathrm{dyn}\, A_n \mid \delta \not\geq$ (vertical notations are also provided but not considered here). This construct is syntactically a schema expression[7]. Semantically, it denotes the subset of the bindings to the plain schema $[x_1 : \mathrm{dyn}\, A_1;\ \ldots;\ x_n : \mathrm{dyn}\, A_n]$, such that in each binding the components behave synchronously, and the dynamic predicate δ is satisfied. The meaning of a dynamic schema is schematically defined as follows, where the A_i's are some sets and dynamic predicates δ will be defined in the next Section:

$$\begin{array}{l}\not\equiv x_1 : \mathrm{dyn}\, A_1;\ \ldots;\ x_n : \mathrm{dyn}\, A_n \mid \delta \not\geq == \\ \quad \{v_1 : \mathrm{dyn}\, A_1;\ \ldots;\ v_n : \mathrm{dyn}\, A_n \mid v_1 \bowtie \ldots \bowtie v_n \land \delta \bullet \\ \qquad \langle x_1 \gneqq v_1;\ \ldots;\ x_n \gneqq v_n \rangle\})\end{array}$$

The synchronicity of the components of a dynamic schema ensures that it is possible to relate their behaviour in the dynamic predicate δ.

For the sake of notational convenience and compatibility with Z, it is possible to provide sets of non-dynamic values in component declarations of dynamic schemes, which will be implicitly lifted to dynamic values. Henceforth, $\not\equiv x : \Re;\ y : \mathrm{dyn}\, \Re \not\geq$ abbreviates $\not\equiv x : \mathrm{dyn}\, \Re;\ y : \mathrm{dyn}\, \Re \not\geq$.

Any dynamic schema implicitly contains a builtin variable, the real-time clock, $\mathsf{clock} \in \mathit{TIME}$. In the linear time model, the variable clock behaves proportional to phase time, otherwise it is just monotonic w.r.t. to phase time (see Section 4.5).

[7] In the *ESZ* version of the Z language, schema expressions can also appear in positions of a schema references or a schema text.

4.2 Dynamic Predicates

A dynamic predicate describes a constraint over a set of free names representing synchronous dynamic values. Below, these names will be called the *dynamic signature* of a dynamic predicate. It is also talked about the *run* of a dynamic signature, which addresses the family of (synchronous) runs of each individual value.

4.2.1 State Predicates

A state predicate allows to describe pointwise constraints on the runs of a dynamic signature. For example, the predicate $\lceil x \geq y \rceil$ is satisfied for those runs where the value of x is continuously greater or equal to that of y at each point in each phase. Thus it is a short notation for $\forall\, i : \mathsf{dom}\, r, t : \mathsf{dom}(ri) \bullet x\, i\, t \geq y\, i\, t$ with the run r. In general, a state predicate $\lceil \phi \rceil$ holds for those (non-empty) runs, where ϕ holds continuously in each phase. Thereby, ϕ may depend on names of the dynamic signature which in this context are interpreted to denote the current value at an observation point (that is, if $x : \mathrm{dyn}\, A$ is a member of the dynamic signature, then in the context of a state predicate, x actually denotes $x\, i\, t$ for the observed phase i at the observed phase-relative time t. It's type then is A.)

Inside of a state predicate, the names of the dynamic signature can be used with a special decoration to refer to their value at the beginning or the end of the run which satisfies the state predicate. For example, $\lceil x_\triangleleft > x_\triangleright \rceil$ specifies that the value of x decreases over a run. In general, a member $x : \mathrm{dyn}\, A$ of the dynamic signature may be used in $\lceil \phi \rceil$ such that $x_\triangleleft$ denotes $\mathsf{left}(x\, 0)$ and $x_\triangleright$ denotes $\mathsf{right}(x\, n)$, where n is the number of the last phase in a run. If the run is infinite $x_\triangleright$ is undefined.

4.2.2 Transition Predicates

A transition predicate describes a phase transition by relating the limes to the right border of a first phase with the limes to the left border of a second phase. It uses name-decoration to denote values at distinct borders. For example, the predicate $\gtrneqq x' = x + 1 \lneqq$ describes those runs with exactly two phases such that the value of x at the left border of the second phase equals to the value of $x + 1$ at the right border of the first phase. This is shown in Figure 2. In general, a member $x : \mathrm{dyn}\, A$ of the dynamic signature may be used in $\gtrneqq \phi \lneqq$ such that x abbreviates $\mathsf{right}(x\, 0)$ and $x'\ \mathsf{left}(x\, 1)$.

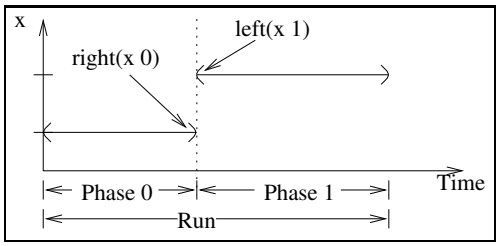


Figure 2: Transition predicates chopping phases

4.2.3 Plain Predicates as Dynamic Predicates

A dynamic predicate may be just a plain Z predicate. For example, the dynamic schema $\not\leq x, y : \mathrm{dyn}\,\Re \mid x = y \not\geq$ describes those runs, where the dynamic values x and y are equal "as a whole" – that means, they are pointwise equal in each phase. The schema $\not\leq x : \mathrm{dis}\,\Re \not\geq$ may be equivalently expressed as $\not\leq x : \mathrm{dyn}\,\Re \mid x \in \mathrm{dis}\,\Re \not\geq$, which shows how constraints of the declarations of components of dynamic schemes are shifted into the property part in a similar way as for plain schemes.

Propositionals have their natural explanation as normal predicates as well. For example, $\not\leq x : \mathrm{dyn}\,\Re \mid \lceil x = 1 \rceil \vee \lceil x = 2 \rceil \not\geq$ describes those runs where x equals either always to 1 or to 2. Note that this is different from $\not\leq x : \mathrm{dyn}\,\Re \mid \lceil x = 1 \vee x = 2 \rceil \not\geq$, where x may change between 1 or 2 from phase to phase.

Plain quantification and schema reference may be used in dynamic predicates as well. Actually, dynamic predicates are syntactically an extension of normal predicates. The priorities for the new operators in conjunction with the plain ones will be defined in Appendix **??**.

4.2.4 Chopping

Dynamic Z provides two notions of chopping. *Transition chopping*, $\delta_1 \nsim \delta_2$, holds for those runs such that for an initial sequence of phases δ_1 holds and for the last phase of this sequence upto the end δ_2. Thus, δ_1 and δ_2 overlap in the last resp. first phase of their satisfaction. For example, $\lceil x \leq c \rceil \nsim \not\geq x = c \not\leq \nsim \lceil x \geq c \rceil$ describes runs where x is continuously less or equal c, than changes its value on a phase transition to exactly c, and then continuously keeps greater then c. This is visualised in Figure 3.

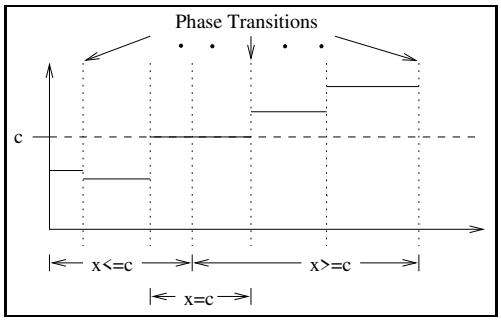


Figure 3: An example trajetory of x according to the temporal formula

Phase chopping, $\delta_1 \diagdown \delta_2$, holds for those runs which can be splitted in a phase or at a phase border such that δ_1 holds up to the splitting point and δ_2 from the splitting point up to the end of the run. It holds $\delta_1 \nsim \not\geq \mathit{true} \not\leq \nsim \delta_2 \Leftrightarrow \delta_1 \diagdown \delta_2$, but the opposite is not true, since the chopping point can be also inside of a phase. This is visualised in Figure 4.

Phase chopping satisfies the basic law known from the DC [1]

$$\lceil \phi \rceil \Leftrightarrow \lceil \phi \rceil \diagdown \lceil \phi \rceil.$$

The two different chopping operators should be used for different design phases. The transition chopping, $\delta_1 \nsim \delta_2$, is mostly used for software design to describe properties of a

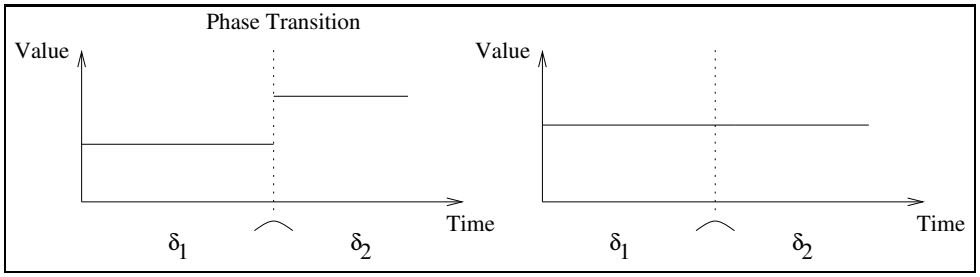


Figure 4: Two examples of phase chopping, one between two phases and the other in one phase

Statechart used to describe a system model. The phase chopping, $\delta_1 \diagdown \delta_2$, in contrast is used to describe real time properties and is thus mostly used for requirement specifications.

4.2.5 Existence and Universality

As usual in interval logics, the *existence* of a sub-run satisfying a dynamic predicate δ, $\Diamond\delta$ (pronounced "somewhere" instead of "eventually", in order to avoid confusion with point logics), is an abbreviation for $true \diagdown \delta \diagdown true$. That a dynamic predicate has to hold for all sub-runs, $\Box\delta$ (pronounced "everywhere"), is an abbreviation for $\neg \Diamond \neg \delta$.

4.2.6 Dynamic Quantification

Dynamic quantification enforces the quantified operands to behave synchronous with the current dynamic signature. Its general form is $\exists\ \sigma \bullet \delta$ resp. $\forall\ \sigma \bullet \delta$, where σ is a dynamic signature. For example, $\not\approx x : \mathrm{dyn} \gtrsim \mid \exists\ t : \mathrm{dyn} \gtrsim \bullet \lceil t = x + 1 \rceil \not\gtrsim$ adds a dynamic variable t temporarily to the dynamic signature, which in each phase equals to the dynamic value of x plus 1.

There is a significant semantical difference between plain quantification and dynamic quantification, even if the first one ranges over dynamic values. A plain quantification, such as $\exists\, t : \mathrm{dyn}\, A \bullet \delta$, just introduces a dynamic value which is not necessarily synchronous with the values in the dynamic signature. Hence it cannot be used in phase and transition predicates.

4.2.7 Phase Number Constraints

Phase number constraints are used as annotations at dynamic predicates to specify the number of phases the predicate should apply to. For example, the state predicate $\lceil\phi\rceil_{=1}$ holds for those runs with exactly one phase in which ϕ holds continuously. $\lceil\phi\rceil_{\geq 0}$ differs from the form without a phase number constraint in that it also holds for runs with 0 phases. State predicates introduced above actually come with default phase number constraints: $\lceil\phi\rceil$ equals to $\lceil\phi\rceil_{>0}$. Like the transition chopping, $\delta_1 \nsim \delta_2$, phase number constrains are used for the software design and shouldn't be mixed up with the phase chopping in one specification.

$\Box_{=2} \gtrsim \phi \not\lesssim$ holds for all runs for which for all sub-runs of length 2 the transition predicate $\gtrsim \phi \not\lesssim$ holds. In contrast $\Box\lceil\phi\rceil_{=2}$ holds if all sub-run of a run contain two phases and ϕ holds

in them. Thus this statement never comes true [8]. In general, ordering relations on natural numbers together with the constants 0, 1, 2 and ∞ can be used in phase number constraints.

4.2.8 Real-Time Constraints

Real-time constraints are of the general form ${}^{t_1}\frac{d}{\delta}{}^{t_2}$, where any of the superscripts is optional. An alternative notation which is more suitable for sequential text (and hence will be used here) is $t_1 - d - t_2\, \delta$. In general, this formula is satisfied for those runs which contain a sub-run which begins at the real-time point t_1 and ends at point t_2, such that δ holds. Thereby, d is the duration of this sub-run, and equals to $t_2 - t_1$. The expressions t_1, t_2 and d must belong to the set *TIME*, that is represent real numbers with the physical unit of seconds.

For example, $-3\mathsf{s}-\lceil x = 0\rceil$ describes those runs which have a sub-run where x is continously zero for 3 seconds. This is equivalent to $true \diagdown \lceil x = 0 \wedge \mathsf{clock}_{\triangleright} - \mathsf{clock}_{\triangleleft} = 3\mathsf{s}\rceil \diagdown true$. Thus real-time constraints are mapped to the builtin variable clock, which is present in every dynamic signature.

In order to fix a real-time constraint to hold from the beginning or end of a run, $t_1 \vdash d - t_2\, \delta$, $t_1 - d \dashv t_2\, \delta$, resp. $t_1 \vdash d \dashv t_2\, \delta$ is used. The following equivalences are satisfied:

$$t_1 - d - t_2\, \delta \equiv true \diagdown t_1 \vdash d - t_2\, \delta \equiv$$
$$t_1 - d \dashv t_2\, \delta \diagdown true \equiv true \diagdown t_1 \vdash d \dashv t_2\, \delta \diagdown true$$

A typical example is $\forall\, t : \mathit{TIME} \bullet \longmapsto t\, \lceil x = 0\rceil \Rightarrow t + 3\mathsf{s} \longmapsto \lceil y = 0\rceil$. This formula holds for those runs where if x equals to zero from the beginning upto some point t, then y equals to 0 from the point t plus 3 seconds to the end of the run[9].

Another typical example is $\exists\, t : \mathit{TIME} \mid t \leq c \bullet \vdash t \dashv \gtrneqq \not{x} = x+1 \not\lessneqq$, which specifies that the addition of 1 to x doesn't durates longer then some constant time c. In order to abbreviate this common case, relations can be directly used in the length part, as in $\vdash \leq c \dashv \gtrneqq \not{x} = x+1 \not\lessneqq$, which means the same as above.

4.3 Embedding of Dynamic Schemes

Dynamic values do not differ from other values in Z, and dynamic schemes are just plain schemes of dynamic values which happen to be synchronous. They are "first-order citizens" of Z, and the whole language repertoire can be used with them.

4.3.1 Declarations and Predicates

Dynamic schemes can appear in declaration or reference contexts of Z wherever a plain schema is allowed. As an example, consider the dynamic schema $D \mathrel{\widehat=} \not\leq x, y : \mathrm{dis}\, \Re \mid \lceil x + y = 0\rceil \not\geq$. Then, in $\not\leq x, y : \mathrm{dyn}\, \Re \mid D \diagdown true \not\geq$, D behaves as a normal schema reference. The names x and y are made synchronous through the context, and (synchronous) initial segments of their

[8] (Ψ Bin nicht sicher, ob die Interpretation zutrifft.)

[9] In a real application we would have to ensure, that the interval after time point t is long enough to contain an interval with minimum length 3_s. This would give another conjunction in the premise of the implication.

runs have to meet the constraint implied by D. This can be explained as usual as textual expansion, yielding $\not\leq x, y : \mathrm{dyn}\,\Re \mid \lceil x + y = 0 \rceil \setminus \mathit{truex}, y \in \mathrm{dis}\,\Re \not\geq$

In $\not\leq D \mid \lceil x > 0 \rceil \not\geq$, D is referred to in the declaration part. The textual expansion yields $\not\leq x, y : \mathrm{dis}\,\Re \mid \lceil x + y = 0 \rceil \wedge \lceil x > 0 \rceil \not\geq$.

4.3.2 Schema Text

Dynamic schemes can be used as schema text in quantors, abstractions, and comprehensions. For example, the set comprehension $\{\not\leq x : \mathrm{dyn}\,A \mid \exists\, c : A \bullet \lceil x = c \rceil \not\geq \bullet x\}$ denotes the set of dynamic values over A which are constant in each run (the characteristic tuple of the comprehension has been made explicite, to illustrate the principle). A name such as $\mathrm{const}\,A$ can be assigned to this set, which then may be used e.g. in a declaration, such as in $\not\leq x : \mathrm{const}\,A \not\geq$. Since the usage of dynamic schemes in set-comprehensions as above is probably one of the most common ones, a syntactic shortcut is provided, the dynamic set-comprehension, $\mathbf{\{}x : \mathrm{dyn}\,A \mid \exists\, c : A \bullet \lceil x = c \rceil\mathbf{\}}$ (here the normal rules for the characteristic tuple take over.)

4.3.3 Expressions

A dynamic schema can be referred in expressions, and then denotes exactly what it means, that is a set of bindings with synchronous components.

An analogous of the θ-expression allows to formate bindings of synchronous values. In $\boldsymbol{\theta}\sigma$, S must denote a schema of dynamic values. A binding is constructed to the signature of the names of S, with the values taken from the context. $\boldsymbol{\theta}\sigma$ is an undefined expressions, iff the values in the context are not synchronous. The predicate $\boldsymbol{\theta}_?\sigma$ tests whether the expression $\boldsymbol{\theta}\sigma$ is defined in the current context.

4.3.4 Schema Expressions

Since dynamic schemes are plain schemes of synchronous dynamic values, all the operations of the schema expression language may be applied to them. For example, $\not\leq x, y : \mathrm{dyn}\,A \mid \delta \not\geq \mathsf{n}\,(y)$ yields a schema containing only the dynamic value x, since y is hidden.

The propositions of the schema expression language applied to schemes of synchronous values do not necessarily result in schemes of synchronous values again. For example, $D_1 \wedge D_2$, with $D_1 \mathrel{\widehat{=}} \not\leq x_1 : \mathrm{dyn}\,A \mid \delta_1 \not\geq$ and $D_2 \mathrel{\widehat{=}} \not\leq x_2 : \mathrm{dyn}\,A \mid \delta_a \not\geq$, yields a schema with components x_1 and x_2 which are not necessarily synchronous in all bindings. If one wants to enforce synchronicity, the operation in question should be embedded in a dynamic schema, such as in $\not\leq D_1 \wedge D_2 \not\geq$, which here works as a kind of filter for synchronicity.

Three new operators allow to drag schemes of dynamic values onto the level of schemes of normal values. In $\downarrow\sigma$, σ must be a schema of synchronous dynamic values, $\not\leq x_1 : \mathrm{dyn}\,A_1;\ \ldots\ ;\ x_n : \mathrm{dyn}\,A_n \not\geq$. The resulting schema is the subset of $[x_1 : A_1;\ \ldots\ ;\ x_n : A_n]$, such that each binding appears at some point in the runs of the original schema. Thus, $\downarrow\sigma$ "forgets" any temporal information. Similarly, $\boldsymbol{\Delta}\sigma$ denotes the subset of $[x_1, x_1' : A_1;\ \ldots\ ;\ x_n, x_n' : A_n]$, such that each binding describes a direct transition between subsequent phases of the runs of the original schema. A typical usage is as in

$State \mathrel{\widehat{=}} \not\leq x : \gtrsim \mid \lceil x \geq 0 \rceil \not\geq$
$Op1 \mathrel{\widehat{=}} [\mathbf{\Delta} State \mid x' = x + 1]$
$Op2 \mathrel{\widehat{=}} [\mathbf{\Delta} State \mid x' = x + 2]$
$Sys \mathrel{\widehat{=}} \not\leq State \mid \Box_{=2} \gtrsim Op1 \vee Op2 \not\lesssim \not\geq$

Thus the $\mathbf{\Delta}\sigma$ schema expression allows to combine plain operational Z specification with temporal state specifications. In accordance to the normal Δ operation, there is also a version $\mathbf{\Xi}\sigma$ with the obvious meaning.

4.4 Integration and Differentiation

Dynamic values represent mathematically sectionally continuous functions. Provided that they range over *linear spaces*, it is possible to integrate dynamic values and to calculate the differential quotient. A linear space supports a notion of addition and of scalar-product.

4.4.1 Integration

The function $\int : \mathrm{dyn}\, A \not\subseteq A$ calculates the integral of a dynamic value according the time notion used in phases (which does not necessarily coincides with real-time). For example, the schema $\not\leq x : \mathrm{dyn}\, \Re \mid \int x \leq c \not\geq$ describes those runs, where the integral of x over the time metric doesn't exceeds a constant $c \in \mathit{TIME}$ (since x is a real-number without units, the integral over time, $\int x$, represents a real-number with the unit of time).

It is possible to integrate also inside of state predicates. For example, the schema $\not\leq x, y : \mathrm{dyn}\, \Re \mid \lceil y = \int x^2 \rceil \not\geq$ describes those runs, where the value of y equals in each point to the square of the integral of x from the beginning of the run upto this point.

Whereas outside of state predicates integration is just an application of a plain function, inside of a state predicate it is a special form with a builtin semantics. In general, in state predicates the expression $\int \omega$ may be undefined, since ω does not necessarily represents a continuous behaviour.

4.4.2 Differentiation

Inside of state predicates the differential quotient of an expression can be referred to by the special expression form **diff** ω, where ω is an expression denoting a value which is element of a linear space.

For example, the schema $\not\leq x, y : \mathrm{dyn}\, \Re \mid \lceil x = y \vee \mathbf{diff}\ abs(x - y) < 0 \rceil \not\geq$ denotes the runs where x and y are either equal or do converge. **diff** ω is a partial expression, which is only defined in a point if ω is differentiable (meaning that it is continuous in this point and the left and right-sided limes at this point coincides).

4.4.3 The Length of a Run

The special expression ℓ refers to the length of a run. It is defined by $\mathsf{right}(\mathsf{clock}, n) - \mathsf{left}(\mathsf{clock}, 0)$ of a run where n is the last phase of the run. According to the DC, ℓ can thought of being just an abbreviation for $\int \mathit{one}$, both outside and inside of phase predicates.

Assuming there is a predefined variable *one* : dyn $\Re$ in each dynamic signature, which has constantly the value 1.

4.5 Specialised Models

Dynamic Z supports specialised models, which can be selected for a specification by inclusion of according axioms.

4.5.1 Discrete Model

In the discrete model, dyn A = dis A holds for any set A. To enforce the discrete model, the schema *Discrete* is included at top-level, which is defined as follows:

$$Discrete[A] \mathrel{\widehat=} [\mathrm{dyn}\, A = \mathrm{dis}\, A]$$

Arbitrary dense real-time constraints can be represented in the discrete as well as in the continuous model – the main pragmatic difference is that the second allows the description of continuous values as well as their *differentiation* and *integration*.

4.5.2 Proportional Real-Time

In the proportional real-time model, the real time clock behaves proportional to phase time. This model is adequate for describing external visible physical behaviour of systems, whereas the default model is suitable for describing internal software behaviour where it is sometimes useful to idealise from real-time. These both models are related to the synchronous resp. asynchronous semantics for Statecharts [2].

In order to enforce that a specification has proportional real-time, one simply includes the schema *LinearRealTime* in the declaration part of a dynamic schema, which is defined as $\not\leq \mathsf{clock} : TIME \bullet \lceil \mathbf{diff}\ \mathsf{clock} = 1 \rceil \not\geq$.

Note that the discrete and the proportional model exclude each other, since in the discrete model the value of clock keeps constant during phases.

A Operational Specification

The *computation*, $\delta_1 \to \delta_2$, is logically similar to $(\delta_1 \nsim \delta_2) \vee (\delta_1 \wedge true_{=\infty})$. However, it describes a further effect, namely that a computation *proceeds*. The point of progress is in the overlapping phase between δ_1 and δ_2. If no such phase exists (that is δ_1 holds infinitely long), the computation deadlocks[10].

The *delayed computation*, $\delta_1 \rightsquigarrow \delta_2$, is an abbreviation for $((\Box\neg\, \delta_1) \setminus \delta_1) \to \delta_2$, which models a computation which idles until it can be performed. For example, $\lceil x < 0 \rceil \rightsquigarrow \gneqq x' = x + 1 \lneqq$ describes a computation which waits until the condition $\lceil x < 0 \rceil$ is satisfied, and then proceeds by adding 1 to x.

[10] (Ψ Nicht ganz klar, ob computation auch ein Prinzip von eagerness enthalten sollte – "so frueh als moeglich".)

The *preemption*, $\delta_1 \nearrow \delta_2$, holds for those runs for which δ_1 and δ_2 simultaneously hold as follows: if δ_2 succeeds as a computation, then δ_1 is abondened from this point on, and for the rest of the run only the continuation of δ_2 has to hold. Otherwise δ_1 has to hold until the end of the run. For example, $\lceil y = \frac{1}{x} \rceil \nearrow (\lceil x = 0 \rceil \leadsto \lceil y = error \rceil)$ describes those runs where y equals to the inverse of x; however, if x equals to 0, then from that point until the end of the run y equals to some constant *error*. This is visualised in Figure **??**.

The *choice*, $\delta_1 \oplus \delta_2$, describes a selection between those computations of its operands which proceed first. In the linear time model (see Section 4.5) it can be characterised by using real-time constraints:

$$\begin{array}{l} \delta_{1,1} \rightarrow \delta_{1,2} \oplus \delta_{2,1} \rightarrow \delta_{2,2} \Leftrightarrow \\ \quad (\forall\, t_1, t_2 : \mathit{TIME} \bullet \\ \qquad \overset{t_1}{\overline{\vdash\!\!\!

D Mathematics of Continuous Systems

In this Appendix, basic notions of the mathematics of continuous systems[11] are defined in and for Z. These definitions are the basis for the formal definition of Dynamic Z, which is given in Appendix C.

D.1 Numbers

The most general number type is defined by the given type *BASENUM*, from which all other number types are derived as subsets. On *BASENUM*, the basic operations addition, multiplication and inverse elements are declared as partial functions. Disjunct elements of *BASENUM* can be denoted by 0, 1, 2, and so on.

$[BASENUM]$

$$
\begin{array}{l}
-, \frac{1}{\mathrm{x}} : BASENUM \not\leqslant BASENUM \\
_ + _, _ * _ : BASENUM \times BASENUM \not\leqslant BASENUM \\
_ < _ : BASENUM \not\geqslant BASENUM
\end{array}
$$

The set of real numbers can be characterised to be the largest subset of *BASENUM* which is an archimedic compound w.r.t. to the basic operations. For a formalisation, archimedic compounds are described for aribitrary sets by the schema below:

ArchimedicCompound$[A]$

$$
\begin{array}{l}
- : A \not\subseteq A \\
\frac{1}{\mathrm{x}} : A \not\leqslant A \\
_ + _, _ * _ : A \times A \not\subseteq A \\
zero, one : A \\
_ < _ : A \not\geqslant A \\
\hline
\forall x, y, z : A \bullet (x + y) + z = x + (y + z) \land x + y = y + x \land \\
\qquad x + zero = x \land x + (-x) = zero \\
\forall x, y, z : A \bullet x * (y * z) = (x * z) * z \land x * y = y * x \land \\
\qquad x * one = x \land (x \in \mathsf{dom}\, \frac{1}{\mathrm{x}} \Rightarrow x * (\frac{1}{\mathrm{x}} x) = one) \\
\forall x, y, z : A \bullet x * (y + z) = (x * y) + (x * z) \\
\mathsf{dom}\, \frac{1}{\mathrm{x}} = \{x : A \mid x \neq zero\} \\
\forall x, y, z : A \bullet (x < y \lor x = y \lor x < y) \land \\
\qquad (x < y \land y < z \Rightarrow x < z) \land \\
\qquad (x < y \Rightarrow x + z < y + z) \land \\
\qquad (x < y \land zero < z \Rightarrow x * z < y * z) \land \\
\qquad (\exists n : (\lambda d : A \bullet d + one)^* \precsim \{zero\} \succsim \bullet x < n)
\end{array}
$$

The diverse number sets can now be formalised as follows:

[11] (Ψ Stehender Begriff? "Analysis" ist fuer den Informatiker nicht eindeutig genug.)

$\Re == \bigcup\{S : \not\approx BASENUM \mid \textbf{let}\ zero == 0;\ one == 1 \bullet ArchimedicCompound[S]\}$
$\mathcal{Q} == \bigcap\{S : \not\approx BASENUM \mid \textbf{let}\ zero == 0;\ one == 1 \bullet ArchimedicCompound[S]\}$
$\gtrsim == (\lambda\, r : \Re \bullet r + 1)^* \precsim \{0\} \gtrsim$
$\gtrsim_1 == \{x : \gtrsim \mid 0 < x\}$
$\gtrsim\!\!\!\!\not= == \gtrsim \cup \{x : \gtrsim \bullet -x\}$

We assume the usual derived operations on numbers.

D.2 Metrics

In order to talk about convergency and continuity w.r.t. to arbitrary sets, the notion of a *metric* is introduced.

D.2.1 Metric Sets

A metric is a set which supports a real-valued *distance* between each pair of its elements. Metric sets are characterised by the schema below:

MetricDistance[*A*]

$$distance : A \times A \not\subseteq \Re$$

$$\forall\, x, y : A \bullet distance(x, y) \geq 0$$
$$\forall\, x, y : A \bullet distance(x, y) = 0 \Leftrightarrow x = y$$
$$\forall\, x, y : A \bullet distance(x, y) = distance(y, x)$$
$$\forall\, x, y, z : A \bullet distance(x, z) \leq distance(x, y) + distance(y, z)$$

Any Z set can be interpreted at least as a degenerated metric by assiging the distance ∞ to pairs of different elements. Thus a standard assignment of a distance function to any set can be assumed, which is modelled by the generic function, dist. For each subset of a set, this function yields the same distance notion:

[*A*]

$$\mathsf{dist} : \not\approx A \not\subseteq A \times A \not\subseteq \Re$$

$$\forall\, M : \not\approx A \bullet \textbf{let}\ d == \mathsf{dist}\ M \bullet MetricDistance[M][d/distance]$$
$$\forall\, M, M' : \not\approx A \mid M \subseteq M' \bullet \mathsf{dist}\ M = M \not\gtrsim \mathsf{dist}\ M'$$

D.2.2 Convergency and Continuity

A series of elements from a metric set *converges* against a *limes*, iff the distance of the elements of the series to the limes converges against 0. If a series converges, the limes is unique. The limes is modelled by a partial function, where a series is represented as a total function from the naturals numbers into a metric set:

$$\text{series}\ A == \gtrsim \not\subseteq A$$

$[A]$

$\lim_{_ \to \infty} _ : \text{series}\, A \not\in A$

$$(\lim_{_ \to \infty} _) = \{s : \text{series}\, A;\ x : A \mid \forall n : \gtrsim \bullet \exists n' : \gtrsim \mid n' > n \bullet \mathsf{dist}\, A\, (s\, n', x) \leq \mathsf{dist}\, A\, (s\, n, x)\}$$

FIXME: da stimmt was nicht

The *limes* of a partial function at a given point which is not necessarily element of its domain is a partial function defined as follows:

$[A, B]$

$\lim_{_ \to _\ _} : A \times (A \not\in B) \not\in B$

$$\begin{array}{l} \forall x : A;\ f : A \not\in B;\ y : B \bullet \\ \quad (x, f) \mapsto y \in (\lim_{_ \to _\ _}) \Leftrightarrow \\ \quad\quad (\exists s : \text{series}(\mathsf{dom}\, f \setminus \{x\}) \bullet s \mapsto x \in (\lim_{_ \to \infty} _)) \wedge \\ \quad\quad (\forall s : \text{series}(\mathsf{dom}\, f \setminus \{x\}) \bullet \\ \quad\quad\quad s \mapsto x \in (\lim_{_ \to \infty} _) \Rightarrow f \circ s \mapsto y \in (\lim_{_ \to \infty} _)) \end{array}$$

A partial function is *continuous* in a point, if the limes at this point exists and is equal to the mapping:

$[A, B]$

$_\ \mathsf{contAt}\ _ : (A \not\in B) \not\ni A$

$$\begin{array}{l} \forall f : A \not\in B;\ x : A \bullet \\ \quad f\ \mathsf{contAt}\ x \Leftrightarrow x \in \mathsf{dom}\, f \wedge (x, f) \mapsto f\, x \in (\lim_{_ \to _\ _}) \end{array}$$

The set of *continuous functions* consists of partial functions which are continuous at each point in the domain:

$$A \not\in\!\!\!\not\in B == \{f : A \not\in B \mid \forall x : \mathsf{dom}\, f \bullet f\ \mathsf{contAt}\ x\}$$

The notion of convergency and continuity introduced above is general enough to apply to functions which map to a "discrete" range, as e.g. a function $\Re \not\in \gtrsim$. The range of such a function must be a singleton set to be continuous, that is, the function must be a constant mapping.

D.2.3 Basic and Derived Metrics

The set of *real numbers* is a metric, where the distance between two numbers is their absolute difference:

$$\mathsf{dist}\, \Re = (\lambda\, x, y : \Re \bullet \mathrm{abs}(x - y))$$

Any *cartesian product* is a metric with a canonical "geometrical" distance derived from the distance of its components. For the case of a product of arity 2, the distance is defined as follows:

$[A, B]$

$$\mathsf{dist}(A \times B) = (\lambda\, x : A \times B;\ y : A \times B \bullet \sqrt{(\mathsf{dist}\, A\, (\mathit{first}\, x, \mathit{first}\, y))^2 + (\mathsf{dist}\, B\, (\mathit{second}\, x, \mathit{second}\, y))^2})$$

This carries over to products with arity greater then 2.

Any *binding set* is a metric. The distance is defined as for products:

...

A *power set* is a metric with the following characterisation[12]:

$[A]$

$$\mathsf{dist}(\not\Subset A) = (\lambda\, x : \not\Subset A;\ y : \not\Subset A \bullet \sqrt{\textstyle\sum_{x \times y}^{(_+_)} (_^2) \circ \mathsf{dist}\, A})$$

D.3 Linear Spaces

Linear spaces are sets which provide a notion of addition and of a scalar-product (**Ψ** Skalarprodukt)[13]. These operations enable the integration and differentiation of functions from a real number into a linear value: for the problem of integration, values can be added and multiplied with a Δ-factor; for the problem of differentiation, values can be substracted and divided by a Δ-factor.

D.3.1 Characterisation of Linear Spaces

A *linear space* is characterised by the schema *LinearOps*:

LinearOps$[A]$

$$_ \oplus _ : A \times A \not\Subseteq A$$
$$_ \otimes _ : \Re \times A \not\Subseteq A$$
$$\mathit{inverse} : A \not\Subseteq A$$
$$\mathit{zero} : A$$

$$\forall\, x, y : A \bullet x \oplus y = y \oplus x$$
$$\forall\, x, y, z : A \bullet (x \oplus y) \oplus z = x \oplus (y \oplus z)$$
$$\forall\, x : A \bullet x \oplus \mathit{zero} = \mathit{zero} \oplus x$$
$$\forall\, x : A \bullet x \oplus (\mathit{inverse}\, x) = \mathit{zero}$$
$$\forall\, r : \Re;\ x, y : A \bullet r \otimes (x \oplus y) = (r \otimes x) \oplus (r \otimes y)$$
$$\forall\, r1, r2 : \Re;\ x : A \bullet (r1 * r2) \otimes x = r1 \otimes (r2 \otimes x)$$
$$\forall\, x : A \bullet 1 \otimes x = x$$

[12] (Ψ Charakterisierung sinnvoll?)

[13] (Ψ Noch nicht ganz klar, ob Beziehung zwischen der Metrik und den Operationen notwendig ist, d.h. *normierte* Raueme.)

The set constructor $\mathrm{lin}\, A$ denotes the empty set if A isn't a linear space, otherwise A itself. The set constructor $\mathrm{linsets}\, A$ denotes the set of linear spaces which are non-empty subsets of the set A:

$$\mathrm{lin}\, A == (\mathbf{if}\ \exists\, \mathit{LinearOps}[A] \bullet \mathit{true}\ \mathbf{then}\ A\ \mathbf{else}\ \not/)$$
$$\mathrm{linsets}\, A == \{A' : \not\ni(\mathrm{lin}\, A) \mid A' \neq \not/\}$$

As for metrics, we assume a unique assignment of operations to any set which is a linear space, modelled by the generic function, *linearOps*:

$$\begin{array}{|l}
\hline
[A] \\
\mathit{linearOps} : \mathrm{linsets}\, A \not\subseteq \mathit{LinearOps}[A] \\
\hline
\forall\, M, M' : \mathrm{linsets}\, A \mid M \subseteq M' \bullet \mathit{linearOps}\ M = \mathit{linearOps}\ M' \\
\hline
\end{array}$$

In order to import the operations of a linear space determined by the function *linearOps* into some local context, the schema *BindLinearOps* is used:

$$\begin{array}{|l}
\hline
\mathit{BindLinearOps}[A] \\
\mathit{LinearOps}[A] \\
\hline
\theta \mathit{LinearOps}[A] = \mathit{linearOps}\ A \\
\hline
\end{array}$$

D.3.2 Basic and Derived Linear Spaces

The real numbers are a linear space:

$$\begin{array}{l}
\mathrm{lin}\, \Re \neq \not/ \wedge \\
(\forall\, \mathit{BindLinearOps}[\Re] \bullet \\
\quad (_ \oplus _) = (\Re \times \Re) \not\ni (_ + _) \in \Re \times \Re \not\subseteq \Re \wedge \\
\quad (_ \otimes _) = (\Re \times \Re) \not\ni (_ * _) \wedge \\
\quad \mathit{inverse} = (-) \wedge \mathit{zero} = 0)
\end{array}$$

Any cartesian product of linear spaces is a linear space again, with operations derived as follows:

$$\begin{array}{|l}
\hline
[A, B] \\
\hline
\mathrm{lin}\, A \neq \not/ \wedge \mathrm{lin}\, B \neq \not/ \Rightarrow \\
\quad \mathrm{lin}(A \times B) \neq \not/ \wedge \\
\quad (\forall\, \mathit{BindLinearOps}[A]_1;\ \mathit{BindLinearOps}[B]_2;\ \mathit{BindLinearOps}[A \times B] \bullet \\
\qquad (\forall\, x_1, y_1 : A;\ x_2, y_2 : B \bullet (x_1, x_2) \oplus (y_1, y_2) = (x_1 \oplus_1 y_1, x_2 \oplus_2 y_2)) \wedge \\
\qquad (\forall\, r : \Re;\ x_1 : A;\ x_2 : B \bullet r \otimes (x_1, x_2) = (r \otimes_1 x_1, r \otimes_2 x_2)) \wedge \\
\qquad (\forall\, x_1 : A;\ x_2 : B \bullet \mathit{inverse}(x_1, x_2) = (\mathit{inverse}_1\, x_1, \mathit{inverse}_2\, x_2)) \wedge \\
\qquad \mathit{zero} = (\mathit{zero}_1, \mathit{zero}_2)) \\
\hline
\end{array}$$

Cartesian products with an arity greater then 2 and binding sets are linear spaces with a similar derivation of the operators. which is specified by reflection:

D.4 Integration and Differentiation over one Variable

We consider integration and differentiation over functions $\Re \not\trianglelefteq \operatorname{lin} A$. The *integral* of such a function in a right-open interval $[a, b)$ is defined as follows:

$[A]$

$$\int\nolimits_{_}^{_}\ _\ _ : \Re \times \Re \times (\Re \not\trianglelefteq \operatorname{lin} A) \not\trianglelefteq A$$

$$\begin{array}{l}
\forall\, a : \Re;\ b : \Re;\ f : \Re \not\trianglelefteq \operatorname{lin} A;\ y : A;\ \mathit{BindLinearOps}[A] \bullet \\
\quad \textbf{let}\ \mathit{sum} == (\lambda\, n : \gtrsim \bullet (\textbf{let}\ \delta == \tfrac{b-a}{n+1} \bullet \\
\qquad\qquad\qquad\qquad (\sum_{0..n}^{(_\oplus_)} \lambda\, i : \gtrsim \bullet \delta \otimes f(a + i * \delta)))) \bullet \\
\quad (a, b, f) \mapsto y \in (\int\nolimits_{_}^{_}\ _\ _) \Leftrightarrow \\
\qquad a < b \wedge [a, b) \subseteq \mathsf{dom}\, f \wedge [a, b) \gtrsim f \in \Re \not\trianglelefteq\ A \wedge \\
\qquad (\mathit{sum}, y) \in (\lim\nolimits_{_ \to \infty}\ _)
\end{array}$$

The *(left-sided) differential quotient* of a function $\Re \not\trianglelefteq \operatorname{lin} A$ in a point is defined as follows:

$[A]$

$$\textbf{diff}_\ _\ _ : \Re \times (\Re \not\trianglelefteq \operatorname{lin} A) \not\trianglelefteq A$$

$$\begin{array}{l}
\forall\, x : \Re;\ f : \Re \not\trianglelefteq \operatorname{lin} A;\ y : A;\ \mathit{BindLinearOps}[A] \bullet \\
\quad \textbf{let}\ \mathit{dq} == (\lambda\, h : \Re \mid h > 0 \wedge x \in \mathsf{dom}\, f \wedge x - h \in \mathsf{dom}\, f \bullet \\
\qquad\qquad\qquad\qquad \tfrac{1}{h} \otimes (f\ x \oplus \mathit{inverse}\, (f\ (x - h)))) \bullet \\
\quad (x, f) \mapsto y \in (\textbf{diff}_\ _\ _) \Leftrightarrow \\
\qquad f\ \mathsf{contAt}\ x \wedge (0, \mathit{dq}) \mapsto y \in (\lim\nolimits_{_ \to _}\ _\ _)
\end{array}$$

References

[1] Chaochen, Z., Hoare, C., and Ravn, A. A calculus of duration. In *Information Processing Letters* (December 1991), vol. 40, pp. 269–276.

[2] Naamad, A., and Harel, D. The statemate semantics of statecharts. *Technical Report* (1996). revised version of "The semantics of Statecharts",1989.